

RAG va vektor ma'lumotlar bazalari

Tabiiy tilni qayta ishlash — 14-ma'ruza

A.A. Abdulali

11:00–12:20 · mavzu №14

7-iyul 2026

Prezentatsiya rejasi

- 1 Kirish: modelga yangi bilimni qidirib beramiz
- 2 LLM cheklovlari va RAG g'oyasi
- 3 RAG jarayoni: qidirish va yaratish
- 4 Vektor embeddinglar va qidiruv
- 5 Vektor ma'lumotlar bazalari
- 6 Xulosa va amaliyotga ko'prik

Takrorlash: o'tgan darsdan ikki savol

1-savol

L13 da fine-tuning modelni vazifaga sozladi. Fine-tuning modelga **yangi fakt-lar** (masalan, kechagi yangilik) ni qo'sha oladimi?

2-savol

LLM bilmagan savolga duch kelsa nima qiladi — "bilmayman" deydimi yoki ishonch bilan xato javob beradimi?

Takrorlash: o'tgan darsdan ikki savol

1-savol

L13 da fine-tuning modelni vazifaga sozladi. Fine-tuning modelga **yangi faktlar** (masalan, kechagi yangilik) ni qo'sha oladimi?

2-savol

LLM bilmagan savolga duch kelsa nima qiladi — "bilmayman" deydimi yoki ishonch bilan xato javob beradimi?

1-javob

Yo'q — fine-tuning bilimni **sozlaydi**, yangi faktni arzon qo'shmaydi. Har yangilik uchun qayta o'qitish qimmat.

2-javob

Ko'pincha ishonch bilan **gallyutsinatsiya** qiladi. Yechim: tashqi manbadan qidirib, javobni unga tayantirish — **RAG**.

Siz bugungi dars oxirida quyidagilarni bajara olasiz:

- ➊ **Tushuntira olasiz:** LLM cheklovlari (gallyutsinatsiya, bilim eskirishi) va RAG nega yordam berishini.
- ➋ **Hisoblay olasiz:** RAG prompt token byudjetini — 3 chunk + instruction = 700 token ($< 1\%$ kontekst oynasi).
- ➌ **Tushuntira olasiz:** vektor embeddinglar bilan semantik qidiruvni (kosinus o'xshashlik, top-k).
- ➍ **Tushuntira olasiz:** vektor ma'lumotlar bazalari (Pinecone, Weaviate) RAG dagi rolini.

L13 dan: fine-tuning. L3/L6 dan: embedding, kosinus o'xshashlik. L4 dan: o'xshashlik qidiruvi (LSH).
Matematikadan: skalyar ko'paytma, ulush.

Bugungi dars rejasi:

Tsiki	Mavzu	Vaqt	Hosil natija	bo'luvchi
1	LLM cheklovlari va RAG g'oyasi	16 daq	gallyutsinatsiya muammosi	
2	RAG jarayoni: qidirish va yaratish	16 daq	prompt = 700 token	
3	Vektor embeddinglar va qidiruv	16 daq	kosinus o'xshashlik	
4	Vektor ma'lumotlar bazalari	16 daq	ANN top-k retrieve	
	Xulosa va 14-amaliyotga ko'rsatmalar	10 daq	Ertaga nima qurishingiz	

Bu dars RAG tizimiga olib keladi

Tashqi bilimni qidirib, LLM javobini unga tayantiramiz. Bugungi prompt = 700 token hisobi ertangi P14 da assert bo'ladi.

Bugungi dars uchun muammo: LLM bilmaganini "to'qiydi"

Faqat LLM qayerda qiynaladi:

Gallyutsinatsiya

LLM bilmagan fakti ishonch bilan **to'qib** chiqaradi — huquq, tibbiyot kabi domenlarda xavfli.

Bilim eskiradi

LLM bilimi o'qitilgan sanada to'xtaydi. Yangi qonun yoki hujjatni bilmaydi; fine-tuning har safar qimmat.

Bugungi yechim:

- 1 Tashqi bilim bazasidan **tegishli** hujjatlarni qidiramiz (retrieve).
- 2 Topilganni promptga **kontekst** qilib qo'shamiz.
- 3 LLM javobni **shu kontekstga** tayantirib yaratadi (generate).

RAG (Retrieval-Augmented Generation): qidiruv + generatsiya. Yangi fakti o'qitishsiz, qidirib qo'shamiz.

Prezentatsiya rejasi

- 1 Kirish: modelga yangi bilimni qidirib beramiz
- 2 LLM cheklovlari va RAG g'oyasi
- 3 RAG jarayoni: qidirish va yaratish
- 4 Vektor embeddinglar va qidiruv
- 5 Vektor ma'lumotlar bazalari
- 6 Xulosa va amaliyotga ko'prik

Intuitsiya: yopiq imtihon va ochiq kitobli imtihon

Faqat LLM — yopiq imtihon: hamma narsa yoddan, bilmaganini taxmin qiladi (gallyutsinatsiya).

RAG — ochiq kitobli imtihon: avval kerakli sahifani **topadi**, keyin o'shanga qarab javob yozadi. Yangi ma'lumotni o'qitishsiz qo'shadi.

Asosiy ark: L13 fine-tuning model **vazn-larini** o'zgartiradi (bilimni sozlaydi); RAG esa **promptga** tashqi fakti qo'shadi (bilimni keltiradi).

$$\mathcal{D}_k = \text{retrieve}(q, \mathcal{C}, k), \quad \hat{y} = \text{LLM}(\text{prompt}(q, \mathcal{D}_k))$$

bunda: q — foydalanuvchi so'rovi; $\mathcal{C}$ — bilim bazasi (hujjat bo'laklari); $\mathcal{D}_k$ — eng tegishli k ta bo'lak; prompt — so'rov + bo'laklar + ko'rsatma; $\hat{y}$ — kontekstga tayangan javob. Model parametrlari o'zgarmaydi — bilim **promptga** qo'shiladi. Yangi hujjat = bazaga qo'shish, qayta o'qitish shart emas.

Keltirib chiqarish: nega qidiruv, fine-tuning emas?

1-qadam. Yangi fakti qo'shish kerak (masalan, bugungi qonun).

Keltirib chiqarish: nega qidiruv, fine-tuning emas?

1-qadam. Yangi fakti qo'shish kerak (masalan, bugungi qonun). **2-qadam.** Fine-tuning bilan: butun modelni qayta o'qitish — qimmat, sekin, har yangilikda takror.

Keltirib chiqarish: nega qidiruv, fine-tuning emas?

1-qadam. Yangi faktni qo'shish kerak (masalan, bugungi qonun). **2-qadam.** Fine-tuning bilan: butun modelni qayta o'qitish — qimmat, sekin, har yangilikda takror. **3-qadam.** RAG bilan: faktni bilim bazasiga **qo'shamiz**, so'rovda uni **qidirib** promptga beramiz — o'qitish yo'q.

Keltirib chiqarish: nega qidiruv, fine-tuning emas?

1-qadam. Yangi fakti qo'shish kerak (masalan, bugungi qonun). **2-qadam.** Fine-tuning bilan: butun modelni qayta o'qitish — qimmat, sekin, har yangilikda takror. **3-qadam.** RAG bilan: fakti bilim bazasiga **qo'shamiz**, so'rovda uni **qidirib** promptga beramiz — o'qitish yo'q. **4-qadam.** Javob manbaga tayanadi $\Rightarrow$ gallyutsinatsiya kamayadi, manbani ko'rsatish mumkin (izohlanuvchanlik).

Xulosa: tez-o'zgaradigan yoki domen bilimi uchun RAG — arzon va yangilanuvchan. Endi prompt byudjetini hisoblaymiz.

Qo'lda misol: faqat LLM noto'g'ri, RAG to'g'ri

LLM 2023-yilgacha o'qitilgan. Savol: "2025-yilgi yangi qoidani tushuntiring."

Faqat LLM:

2025 faktini **bilmaydi** $\Rightarrow$ to'g'ri javob ehtimoli ≈ 0 ,
ammo ishonch bilan to'qiydi.

RAG: 2025 hujjatini bazadan topadi,
promptga qo'shadi $\Rightarrow$ javob **manbaga**
tayanadi, to'g'ri bo'ladi.

Sizning vazifangiz: qaysi holatda RAG kerak?

Savol — 60 soniya

Quyidagidan qaysi biri uchun RAG (fine-tuning emas) mos: (a) modelga rasmiy, muloyim uslubni o'rgatish; (b) kompaniyaning bugungi ichki hujjatlariga javob berish?

Sizning vazifangiz: qaysi holatda RAG kerak?

Savol — 60 soniya

Quyidagidan qaysi biri uchun RAG (fine-tuning emas) mos: (a) modelga rasmiy, muloyim uslubni o'rgatish; (b) kompaniyaning bugungi ichki hujjatlariga javob berish?

Javob

(b) — tez-o'zgaradigan, tashqi faktlar uchun RAG. (a) uslub/format uchun fine-tuning mosroq (bu xulq-atvor, fakt emas).

Qoida: **xulq/uslub** → fine-tuning; **fakt/bilim** → RAG.

Kod $\leftrightarrow$ formula: RAG skeleti

```
def rag_answer(query, store, llm, k=3):  
    # 1) qidirish (retrieve)  
    docs = store.search(query, k=k)  
    # 2) prompt yig'ish  
    context = "\n".join(docs)  
    prompt = f"Kontekst:\n{context}\n\nSavol: {  
        query}"  
    # 3) javob yaratish (generate)  
    return llm.generate(prompt)
```

Kod $\rightarrow$ formula:

Kod	Formula
search	$\text{retrieve}(q, \mathcal{C}, k)$
prompt	$\text{prompt}(q, \mathcal{D}_k)$
generate	$\hat{y} = \text{LLM}(\cdot)$

P14 da m14 aynan shu uch qadamni quradi.

RAG tuzog'i: yomon qidiruv = yomon javob

Garbage in, garbage out

Agar retrieve tegishli bo'lmagan hujjatlarni qaytarsa, LLM ham noto'g'ri javob beradi. RAG sifati **qidiruv sifatiga** bog'liq.

Yechim: yaxshi embedding + to'g'ri chunking + mos k . Keyingi sikllarda shularni ko'ramiz.

Prezentatsiya rejasi

- 1 Kirish: modelga yangi bilimni qidirib beramiz
- 2 LLM cheklovlari va RAG g'oyasi
- 3 RAG jarayoni: qidirish va yaratish**
- 4 Vektor embeddinglar va qidiruv
- 5 Vektor ma'lumotlar bazalari
- 6 Xulosa va amaliyotga ko'prik

Intuitsiya: kontekst oynasiga faqat keraklisini joylaymiz

LLM kontekst oynasi cheklangan (masalan 128k token) va har token **narx**. Butun bazani joylab bo'lmaydi.

Shuning uchun faqat **eng tegishli** bir nechta bo'lakni (top-k) qidirib, promptga qo'shamiz — qisqa, arzon, aniq.

Prompt = ko'rsatma + k ta bo'lak + so'rov. Token byudjetini hisoblash — amaliy zarurat (narx va tezlik).

RAG prompt byudjeti: rasmiy ta'rif

$$T_{\text{prompt}} = k \cdot T_{\text{chunk}} + T_{\text{instr}}, \quad \text{ulush} = \frac{T_{\text{prompt}}}{T_{\text{window}}}$$

bunda: k — retrieved bo'laklar soni; T_{chunk} — bir bo'lak token soni; T_{instr} — ko'rsatma (instruction) tokenlari; T_{window} — LLM kontekst oynasi (masalan 128000). Maqsad: byudjet kontekst oynasiga sig'sin va narx past bo'lsin.

Keltirib chiqarish: prompt nechta tokendan iborat?

1-qadam. Retrieve k ta bo'lak qaytaradi, har biri $\approx T_{\text{chunk}}$ token.

Keltirib chiqarish: prompt nechta tokendan iborat?

1-qadam. Retrieve k ta bo'lak qaytaradi, har biri $\approx T_{\text{chunk}}$ token. **2-qadam.** Bo'laklar tokenlari: $k \cdot T_{\text{chunk}}$.

Keltirib chiqarish: prompt nechta tokendan iborat?

1-qadam. Retrieve k ta bo'lak qaytaradi, har biri $\approx T_{\text{chunk}}$ token. **2-qadam.** Bo'laklar tokenlari: $k \cdot T_{\text{chunk}}$. **3-qadam.** Ustiga ko'rsatma (instruction) qo'shamiz: $+ T_{\text{instr}}$.

Keltirib chiqarish: prompt nechta tokendan iborat?

1-qadam. Retrieve k ta bo'lak qaytaradi, har biri $\approx T_{\text{chunk}}$ token. **2-qadam.** Bo'laklar tokenlari: $k \cdot T_{\text{chunk}}$. **3-qadam.** Ustiga ko'rsatma (instruction) qo'shamiz: $+ T_{\text{instr}}$. **4-qadam.** Jami $T_{\text{prompt}} = k \cdot T_{\text{chunk}} + T_{\text{instr}}$; uni oynaga nisbatan ulush sifatida baholaymiz.

Xulosa: byudjet k va bo'lak hajmiga chiziqli bog'liq. Endi aniq sonlar bilan hisoblaymiz.

Qo'lda hisob: RAG prompt = 700 token

Kontekst oynasi $T_{\text{window}} = 128000$ token.

Hisob:

$$T_{\text{prompt}} = 3 \times 200 + 100 = \mathbf{700}$$
$$\text{ulush} = \frac{700}{128000} \approx 0.0055 < 1\%$$

700 token — oynaning 1% idan kam.
Javob halliysinatsiya o'rniga kontekstga tayanadi; narx minimal.

Ertangi P14 birinchi assert:
`assert prompt_tokens == 700`
`assert retrieved_docs == 3`
Ma'ruza L14 [l2]-slayd

Sizning vazifangiz: 5 bo'lak uchun byudjet

Savol — 60 soniya

$k = 5$, $T_{\text{chunk}} = 200$, $T_{\text{instr}} = 100$:

$$T_{\text{prompt}} = 5 \times 200 + 100 = ?$$

Sizning vazifangiz: 5 bo'lak uchun byudjet

Savol — 60 soniya

$k = 5$, $T_{\text{chunk}} = 200$, $T_{\text{instr}} = 100$:

$$T_{\text{prompt}} = 5 \times 200 + 100 = ?$$

Javob

$$T_{\text{prompt}} = 1000 + 100 = \mathbf{1100} \text{ token}; \quad \text{ulush} = \frac{1100}{128000} \approx 0.86\%$$

Ko'proq bo'lak $\Rightarrow$ ko'proq token va narx, ammo qamrov kengroq. k ni muvozanatlash kerak (qamrov vs narx).

Kod ↔ formula: prompt byudjeti

```
1 def prompt_tokens(k, t_chunk, t_instr):
2     return k * t_chunk + t_instr
3
4 T = prompt_tokens(k=3, t_chunk=200, t_instr=100)
5 window = 128000
6 print(T)                # 700
7 print(T / window)       # 0.0055 (< 1%)
```

Kod → formula:

Kod	Formula
$k * t_chunk$	$k \cdot T_{\text{chunk}}$
$+ t_instr$	$+ T_{\text{instr}}$
T / window	ulush

P14 da m14 promptni shu byudjet bilan yig'adi (token nazorati).

RAG jarayoni tuzog'i: juda ko'p kontekst zarar qiladi

"Lost in the middle"

Juda ko'p bo'lak qo'shsak: narx oshadi va model o'rtadagi muhim ma'lumotni "yo'qotadi" (e'tibor suyuladi). Ko'p = har doim yaxshi emas.

Yechim: kichik k (masalan 3–5), yaxshi tartiblash (reranking), aniq chunking.

Prezentatsiya rejasi

- 1 Kirish: modelga yangi bilimni qidirib beramiz
- 2 LLM cheklovlari va RAG g'oyasi
- 3 RAG jarayoni: qidirish va yaratish
- 4 Vektor embeddinglar va qidiruv**
- 5 Vektor ma'lumotlar bazalari
- 6 Xulosa va amaliyotga ko'prik

Intuitsiya: ma'no bo'yicha qidiramiz, so'z bo'yicha emas

Kalit so'z qidiruvi "mashina" va "avtomobil" ni boshqa deb biladi. **Vektor qidiruv** ularni **yaqin** deb topadi — ma'no bo'yicha.

So'rov va hujjatlar embedding (L3/L6) ga aylantiriladi; eng **yaqin** vektorlar (kosinus o'xshashlik) qaytariladi.

Embedding L3/L6 da o'rganilgan — bu yerda **qidiruv** uchun qayta ishlatiladi. Semantik o'xshashlik = kosinus.

Semantik qidiruv: rasmiy ta'rif

$$\text{sim}(q, d) = \frac{\mathbf{e}_q \cdot \mathbf{e}_d}{\|\mathbf{e}_q\| \|\mathbf{e}_d\|}, \quad \mathcal{D}_k = \underset{d \in \mathcal{C}}{\text{top-k}} \text{ sim}(q, d)$$

bunda: $\mathbf{e}_q, \mathbf{e}_d$ — so'rov va hujjat embeddinglari; sim — kosinus o'xshashlik (L3); $\mathcal{D}_k$ — eng yuqori k ta o'xshash hujjat. Embedding modeli (masalan `sentence-transformers`) matnni vektorga aylantiradi; qidiruv — eng yaqin vektorlarni topish.

Keltirib chiqarish: nega kosinus o'xshashlik?

1-qadam. Matnlarni embedding fazosida vektor sifatida ifodalaymiz (L3/L6) — yaqin ma'no, yaqin vektor.

Keltirib chiqarish: nega kosinus o'xshashlik?

1-qadam. Matnlarni embedding fazosida vektor sifatida ifodalaymiz (L3/L6) — yaqin ma'no, yaqin vektor. **2-qadam.** Yaqinlikni o'lchash kerak: skalyar ko'paytma uzunlikka bog'liq $\Rightarrow$ normallashtiramiz.

Keltirib chiqarish: nega kosinus o'xshashlik?

1-qadam. Matnlarni embedding fazosida vektor sifatida ifodalaymiz (L3/L6) — yaqin ma'no, yaqin vektor. **2-qadam.** Yaqinlikni o'lchash kerak: skalyar ko'paytma uzunlikka bog'liq $\Rightarrow$ normallashtiramiz. **3-qadam.** Normallangan skalyar ko'paytma = **kosinus** o'xshashlik $([-1, 1])$, uzunlikdan mustaqil.

Xulosa: kosinus — yo'nalish (ma'no) o'xshashligi. Eng yuqori k tasi retrieve qilinadi. Endi qo'lda hisoblaymiz.

Qo'lda misol: kosinus o'xshashlik = $2/3$

$$\mathbf{e}_q = (1, 1, 1, 0), \quad \mathbf{e}_d = (1, 1, 0, 1) \text{ (lug'at: nlp, qonun, hujjat, sud).}$$

Kosinus:

$$\begin{aligned}\mathbf{e}_q \cdot \mathbf{e}_d &= 1 + 1 + 0 + 0 = 2 \\ \|\mathbf{e}_q\| &= \|\mathbf{e}_d\| = \sqrt{3} \\ \text{sim} &= \frac{2}{\sqrt{3} \cdot \sqrt{3}} = \frac{2}{3} \approx \mathbf{0.667}\end{aligned}$$

L3 dagi kosinus o'xshashlik — endi RAG qidiruvida. 0.667 — ancha yaqin; bu hujjat top-k ga tushadi.

Sizning vazifangiz: boshqa hujjat o'xshashligi

Savol — 60 soniya

$$\mathbf{e}_q = (1, 1, 1, 0), \quad \mathbf{e}_d = (1, 0, 0, 0):$$

$$\text{sim} = \frac{\mathbf{e}_q \cdot \mathbf{e}_d}{\sqrt{3} \cdot \sqrt{1}} = ?$$

Sizning vazifangiz: boshqa hujjat o'xshashligi

Savol — 60 soniya

$$\mathbf{e}_q = (1, 1, 1, 0), \quad \mathbf{e}_d = (1, 0, 0, 0):$$

$$\text{sim} = \frac{\mathbf{e}_q \cdot \mathbf{e}_d}{\sqrt{3} \cdot \sqrt{1}} = ?$$

Javob

$$\mathbf{e}_q \cdot \mathbf{e}_d = 1; \quad \text{sim} = \frac{1}{\sqrt{3}} \approx \mathbf{0.577}$$

Birinchi hujjatdan (0.667) kamroq o'xshash $\Rightarrow$ top-k da pastroq o'rin egallaydi.

Kod $\leftrightarrow$ formula: semantik qidiruv

```
1 import numpy as np
2
3 def cosine(a, b):
4     return a @ b / (np.linalg.norm(a) * np.linalg
5                       .norm(b))
6
7 def top_k(q_vec, doc_vecs, k=3):
8     sims = [cosine(q_vec, d) for d in doc_vecs]
9     order = np.argsort(sims)[::-1]
10    return order[:k]
```

Kod $\rightarrow$ formula:

Kod	Formula
cosine	$\text{sim}(q, d)$
argsort	top-k

Embedding modeli matnni
q_vec/doc_vecs ga aylanti-
radi (sentence-transformers).

Embedding tuzog'i: chunking muhim

Noto'g'ri bo'lak hajmi

Juda katta bo'lak — embedding ma'noni "suyultiradi"; juda kichik bo'lak — kontekst yo'qoladi. Qidiruv sifati chunking ga bog'liq.

Yechim: mazmunan to'liq, o'rtacha hajmli bo'laklar (masalan ahzo/paragraf), biroz ustma-ust (overlap) bilan.

Prezentatsiya rejasi

- 1 Kirish: modelga yangi bilimni qidirib beramiz
- 2 LLM cheklovlari va RAG g'oyasi
- 3 RAG jarayoni: qidirish va yaratish
- 4 Vektor embeddinglar va qidiruv
- 5 Vektor ma'lumotlar bazalari**
- 6 Xulosa va amaliyotga ko'prik

Intuitsiya: millionlab vektorni tez qidirish kerak

Bilim bazasi katta bo'lsa (millionlab bo'lak), har so'rovda hammasi bilan kosinus hisoblash **sekin**.

Vektor ma'lumotlar bazasi (Pinecone, Weaviate, FAISS) vektorlarni indekslaydi va **taxminiy** eng yaqinlarni (ANN) tez topadi.

ANN (approximate nearest neighbor): aniqlikning ozgina qismini tezlikka almashtiradi — millionlab vektorda millisoniyalarda qidiradi.

$$\text{upsert} : \mathcal{C} \rightarrow \text{indeks } \mathcal{I}, \quad \text{query}(q, k) : \mathcal{I} \rightarrow \mathcal{D}_k \text{ (ANN)}$$

bunda: $\mathcal{I}$ — vektor indeksi (masalan HNSW grafi); upsert — hujjat vektorlarini bazaga qo'shish; query — so'rov vektoriga eng yaqin k tasini **taxminan** topish. To'liq qidiruv $O(n)$; ANN indeks $\approx O(\log n)$ — katta bazada hal qiluvchi tezlik.

Keltirib chiqarish: nega taxminiy qidiruv (ANN)?

1-qadam. Aniq eng yaqinni topish = har bir vektor bilan taqqoslash = $O(n)$ har so'rovda.

Keltirib chiqarish: nega taxminiy qidiruv (ANN)?

- 1-qadam.** Aniq eng yaqinni topish = har bir vektor bilan taqqoslash = $O(n)$ har so'rovda.
2-qadam. Millionlab vektorda bu juda sekin (real vaqtga yaramaydi).

Keltirib chiqarish: nega taxminiy qidiruv (ANN)?

1-qadam. Aniq eng yaqinni topish = har bir vektor bilan taqqoslash = $O(n)$ har so'rovda.

2-qadam. Millionlab vektorda bu juda sekin (real vaqtga yaramaydi). **3-qadam.** Indeks (HNSW grafi yoki klaster) faqat **nomzod** vektorlarni ko'radi $\Rightarrow \approx O(\log n)$.

Keltirib chiqarish: nega taxminiy qidiruv (ANN)?

1-qadam. Aniq eng yaqinni topish = har bir vektor bilan taqqoslash = $O(n)$ har so'rovda.

2-qadam. Millionlab vektorda bu juda sekin (real vaqtga yaramaydi). **3-qadam.** Indeks (HNSW grafi yoki klaster) faqat **nomzod** vektorlarni ko'radi $\Rightarrow \approx O(\log n)$. **4-qadam.** Ozgina aniqlik yo'qoladi (taxminiy), ammo tezlik ulkan — amalda maqbul almashinish.

Xulosa: vektor DB tezlik uchun aniqlikning kichik qismini beradi. Endi sonlar bilan ko'ramiz.

Qo'lda misol: 10000 bo'lak, top-3 retrieve

So'rovga eng yaqin $k = 3$ bo'lak kerak.

Taqqoslash:

- Brute-force: 10000 kosinus hisob har so'rovda ($O(n)$).
- ANN (HNSW): $\approx \log_2(10000) \approx 13$ qadam ($O(\log n)$).

$$\frac{10000}{13} \approx \mathbf{770} \times \text{tezroq (taxminan)}$$

Kichik bazada brute-force ham yetadi; millionlab bo'lakda ANN shart. Ikkalasi ham top-3 qaytaradi.

Sizning vazifangiz: 1 million vektor uchun ANN qadami

Savol — 60 soniya

$n = 1000000$ vektor. ANN taxminan $\log_2 n$ qadam.

$\log_2(10^6) \approx ?$ ($2^{20} \approx 10^6$)

Sizning vazifangiz: 1 million vektor uchun ANN qadami

Savol — 60 soniya

$n = 1000000$ vektor. ANN taxminan $\log_2 n$ qadam.

$\log_2(10^6) \approx ?$ ($2^{20} \approx 10^6$)

Javob

$\log_2(10^6) \approx \mathbf{20}$ qadam (brute-force 10^6 o'rniga)

Million vektorda ANN ≈ 20 taqqoslash — $50000\times$ tezroq. Mana shu uchun vektor DB kerak.

Kod $\leftrightarrow$ formula: vektor DB (upsert/query)

```
1 # soddalashtirilgan vektor DB interfeysi
2 index.upsert(ids, vectors)          # bazaga qo'shish
3
4 # so'rovga eng yaqin k ta (ANN)
5 hits = index.query(query_vec, top_k=3)
6 docs = [store[h.id] for h in hits]
```

Kod $\rightarrow$ formula:

Kod	Formula
upsert	$\mathcal{C} \rightarrow \mathcal{I}$
query	$\mathcal{I} \rightarrow \mathcal{D}_k$

Pinecone/Weaviate/FAISS shu interfeysni beradi. P14 da m14 FAISS bilan ishlaydi.

Vektor DB tuzog'i: indeks va embedding mos bo'lishi shart

Nomuvofiq embedding

Upsert va query bir xil embedding modeli bilan bo'lishi kerak. Boshqa model $\Rightarrow$ vektorlar mos kelmaydi $\Rightarrow$ qidiruv ma'nosiz.

Yechim: bir embedding modelni qat'iy saqlang; o'lcham (dim) indeks bilan mos bo'lsin.

Prezentatsiya rejasi

- 1 Kirish: modelga yangi bilimni qidirib beramiz
- 2 LLM cheklovlari va RAG g'oyasi
- 3 RAG jarayoni: qidirish va yaratish
- 4 Vektor embeddinglar va qidiruv
- 5 Vektor ma'lumotlar bazalari
- 6 Xulosa va amaliyotga ko'prik

O'zbek tilida: RAG va huquqiy matnlar (lex.uz)

1. Nega RAG kerak:

lex.uz qonunchilik hujjatlari tez yangilanadi va aniqlik talab qiladi. LLM ularni yoddan bilmaydi yoki eskirgan biladi.

Xavf

Huquq/tibbiyot kabi domenda gallyutsinatsiya **xavfli** — noto'g'ri qonun moddasi jiddiy oqibatga olib keladi.

2. RAG yechimi:

Hujjatni lex.uz bazasidan qidirib, javobni unga tayantiramiz va **manbani ko'rsatamiz**.

O'zbek tilida sifatli embedding (ko'p tilli model) muhim — WordPiece qo'shimchalarni kessa (L13 [M]), qidiruv sifati tushadi.

Sintez: faqat LLM, fine-tuning va RAG

Jihat	Fine-tuning (L13)	RAG (L14)
O'zgartiradi	model vaznlari	prompt (tashqi kontekst)
Yangi fakt	qimmat, qayta o'qitish	arzon, bazaga qo'shish
Gallyutsinatsiya	qisman	kamayadi (manbaga tayanadi)
Eng mos	xulq/uslub	fakt/bilim, tez-o'zgaruvchi

Asosiy g'oya

RAG: embedding bilan qidir (top-k) → promptga qo'sh → javob yarat. Bilim promptga keladi, modelga emas. Vektor DB katta bazada tezlik beradi.

Seminal manba: Lewis va boshqalar (2020)

Lewis, P., Perez, E., Piktus, A., va boshq. (2020). **Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.** *NeurIPS* 33.

Ahamiyati:

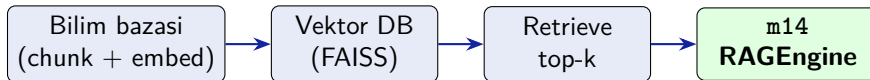
- Qidiruv va generatsiyani birlashtirib, parametrik bilimni tashqi manba bilan kengaytirdi.
- Bilimni yangilash uchun qayta o'qitish shart emasligini ko'rsatdi.
- Zamonaviy LLM ilovalarining (chatbot, yordamchi) asosiga aylandi.

Muhokama savoli (P14 gacha o'ylang)

O'zbek huquqiy hujjatlari uchun RAG qurganda, chunking (bo'lak hajmi) va embedding modelini qanday tanlaysiz? Qo'shimchalar qidiruvga qanday ta'sir qiladi?

- ① ✓ **Tushuntirdik:** LLM cheklovlari (gallyutsinatsiya, bilim eskirishi) va RAG yechimi.
- ② ✓ **Hisobladik:** RAG prompt byudjeti — $3 \times 200 + 100 = 700$ token ($< 1\%$ oyna).
- ③ ✓ **Tushuntirdik:** vektor embedding va kosinus o'xshashlik bilan semantik qidiruv (2/3).
- ④ ✓ **Tushuntirdik:** vektor ma'lumotlar bazalari (ANN) va ularning RAG dagi roli.

Ertaga P14: o'zbek hujjatlari uchun RAG quramiz



P14 da qurasiz:

- m14 RAGEngine: chunk + embed + FAISS + generate
- top-k retrieve va prompt yig'ish
- javobni kontekstga tayantirish

Birinchi assert (bu slayd [I2]):

```
assert prompt_tokens == 700  
assert retrieved_docs == 3
```

*# Ma'ruza L14 [I2]-slayd bilan
solishtiring (RAG token hisobi)*

- ❶ Lewis, P., va boshq. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS 33.
- ❷ Karpukhin, V., va boshq. (2020). *Dense Passage Retrieval for Open-Domain Question Answering*. EMNLP 2020.
- ❸ Johnson, J., Douze, M., Jegou, H. (2019). *Billion-scale Similarity Search with GPUs (FAISS)*. IEEE Big Data.
- ❹ Malkov, Y., Yashunin, D. (2018). *Efficient and Robust ANN Search using HNSW*. IEEE TPAMI.
- ❺ sentence-transformers va faiss hujjatlari.

Ilova S1: chunking strategiyalari

Hujjatni bo'laklarga ajratishning keng tarqalgan usullari:

- **Sobit hajm:** har N token (oddiy, ammo gapni kesishi mumkin).
- **Gap/paragraf bo'yicha:** mazmunan to'liq, tabiiy chegaralar.
- **Overlap bilan:** bo'laklar biroz ustma-ust — chegaradagi kontekst yo'qolmaydi.

bunda: O'zbek huquqiy matnida modda/band bo'yicha bo'lish ko'pincha eng mosi — har bo'lak mustaqil ma'noga ega bo'ladi.

Ilova S2: HNSW indeks (qisqacha)

HNSW (Hierarchical Navigable Small World) — ANN uchun ko'p qatlamli graf:

- Yuqori qatlamlar — siyrak, uzoq "sakrashlar" (tez yo'naltirish).
- Quyi qatlamlar — zich, aniq qo'shnilar.
- Qidiruv yuqoridan boshlanib, pastga "tushadi" — $\approx O(\log n)$.

bunda: Aniqlik/tezlik almashinishi ef va M parametrlari bilan sozlanadi — katta ef aniqroq, ammo sekinroq.